

//IPPR Google collab code

1)Write a code for obtaining negative of the image. Input: Gray scale 8 bit image Expected output: Gray scale 8 bit image

```
# Import necessary libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
from PIL import Image
import io

# Upload image
uploaded = files.upload()

# Load the image in grayscale mode
for file in uploaded.keys():
    image_path = file
    img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

    if img is None:
        print("Failed to load image.")
    else:
        # Get negative image: 255 - pixel_value
        negative_img = 255 - img

        # Display original and negative image using matplotlib
        plt.figure(figsize=(10, 4))
        plt.subplot(1, 2, 1)
        plt.title("Original Image")
        plt.imshow(img, cmap='gray')
        plt.axis('off')

        plt.subplot(1, 2, 2)
        plt.title("Negative Image")
        plt.imshow(negative_img, cmap='gray')
        plt.axis('off')

        plt.tight_layout()
        plt.show()
```

```
# Save negative image
cv2.imwrite("negative_image.png", negative_img)
files.download("negative_image.png")
```

2)Write a code for obtaining negative of the image. Input: 24 bit color image Expected output: 24 bit color image

```
# Import necessary libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Upload a 24-bit color image
uploaded = files.upload()

# Read and process the uploaded image
for file_name in uploaded.keys():
    # Read the image in color (24-bit = 8 bits per R, G, B channel)
    img = cv2.imread(file_name)

    if img is None:
        print("Failed to load image.")
    else:
        # Convert from BGR (OpenCV default) to RGB for display
        img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

        # Get the negative image: 255 - pixel_value for each channel
        negative_img = 255 - img_rgb

        # Display original and negative image using matplotlib
        plt.figure(figsize=(10, 4))

        plt.subplot(1, 2, 1)
        plt.title("Original Image")
        plt.imshow(img_rgb)
```

```

plt.axis('off')

plt.subplot(1, 2, 2)
plt.title("Negative Image")
plt.imshow(negative_img)
plt.axis('off')

plt.tight_layout()
plt.show()

# Convert back to BGR before saving with OpenCV
negative_bgr = cv2.cvtColor(negative_img, cv2.COLOR_RGB2BGR)
cv2.imwrite("negative_color_image.png", negative_bgr)

# Download the negative image
files.download("negative_color_image.png")

```

3)Write a code for obtaining CMY equivalent of the image Input: 24 bit RGB image Expected output: 24 bit CMY image

```

# Import required libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Upload a 24-bit RGB image
uploaded = files.upload()

# Read and convert to CMY
for file_name in uploaded.keys():
    # Load image using OpenCV (in BGR format)
    bgr_img = cv2.imread(file_name)

    if bgr_img is None:
        print("Failed to load image.")
    else:
        # Convert BGR to RGB

```

```

rgb_img = cv2.cvtColor(bgr_img, cv2.COLOR_BGR2RGB)

# Normalize RGB to [0, 1]
rgb_normalized = rgb_img / 255.0

# Convert to CMY: C = 1 - R, M = 1 - G, Y = 1 - B
cmy_normalized = 1 - rgb_normalized

# Scale CMY back to 0-255 and convert to uint8
cmy_img = (cmy_normalized * 255).astype(np.uint8)

# Display original and CMY image
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.title("Original RGB Image")
plt.imshow(rgb_img)
plt.axis('off')

plt.subplot(1, 2, 2)
plt.title("CMY Image")
plt.imshow(cmy_img)
plt.axis('off')

plt.tight_layout()
plt.show()

# Convert back to BGR for saving with OpenCV
cmy_bgr = cv2.cvtColor(cmy_img, cv2.COLOR_RGB2BGR)
cv2.imwrite("cmy_image.png", cmy_bgr)

# Download the CMY image
files.download("cmy_image.png")

```

4) Write a code for obtaining edge detection of the Image using Sobel operator Input: 24 bit RGB image Expected Output: Segmented Image

```

# Import necessary libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Upload a 24-bit RGB image
uploaded = files.upload()

# Read and process the image
for file_name in uploaded.keys():
    # Load image in color
    img_bgr = cv2.imread(file_name)

    if img_bgr is None:
        print("Failed to load image.")
    else:
        # Convert to grayscale (Sobel works on single channel)
        gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)

        # Apply Sobel operator in X and Y direction
        sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
        sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

        # Calculate gradient magnitude
        sobel_combined = cv2.magnitude(sobel_x, sobel_y)

        # Normalize to 8-bit range
        sobel_output = cv2.normalize(sobel_combined, None, 0, 255,
cv2.NORM_MINMAX)
        sobel_output = sobel_output.astype(np.uint8)

        # Display original and edge-detected image
        plt.figure(figsize=(10, 4))

        plt.subplot(1, 2, 1)
        plt.title("Original Image")
        plt.imshow(cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB))
        plt.axis('off')

```

```

plt.subplot(1, 2, 2)
plt.title("Sobel Edge Detection")
plt.imshow(sobel_output, cmap='gray')
plt.axis('off')

plt.tight_layout()
plt.show()

# Save and download the result
cv2.imwrite("sobel_edge_output.png", sobel_output)
files.download("sobel_edge_output.png")

```

5)Write a code for obtaining edge detection of the Image using Prewitt operator Input: 24 bit RGB image Expected Output: Segmented Image

```

# Import necessary libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Upload a 24-bit RGB image
uploaded = files.upload()

# Read and apply Prewitt edge detection
for file_name in uploaded.keys():
    # Load the image
    img_bgr = cv2.imread(file_name)

    if img_bgr is None:
        print("Failed to load image.")
    else:
        # Convert to grayscale
        gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)

        # Define Prewitt kernels
        kernel_x = np.array([[ -1,  0,  1],
                             [ -1,  0,  1],

```

```

        [-1,  0,  1]], dtype=np.float32)

kernel_y = np.array([[ 1,  1,  1],
                     [ 0,  0,  0],
                     [-1, -1, -1]], dtype=np.float32)

# Apply kernels using filter2D
prewitt_x = cv2.filter2D(gray, -1, kernel_x)
prewitt_y = cv2.filter2D(gray, -1, kernel_y)

# Combine both directions
prewitt_combined = cv2.magnitude(prewitt_x.astype(np.float32),
prewitt_y.astype(np.float32))
prewitt_combined = cv2.normalize(prewitt_combined, None, 0, 255,
cv2.NORM_MINMAX).astype(np.uint8)

# Display results
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.title("Original Image")
plt.imshow(cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB))
plt.axis('off')

plt.subplot(1, 3, 2)
plt.title("Prewitt - X Direction")
plt.imshow(prewitt_x, cmap='gray')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.title("Prewitt Edge Detection")
plt.imshow(prewitt_combined, cmap='gray')
plt.axis('off')

plt.tight_layout()
plt.show()

# Save and download the result
cv2.imwrite("prewitt_edge_output.png", prewitt_combined)
files.download("prewitt_edge_output.png")

```

6)Write a code for obtaining edge detection of the Image using Canny operator Input: 24 bit RGB image Expected Output: Segmented Image

```
# Import required libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Upload a 24-bit RGB image
uploaded = files.upload()

# Read and apply Canny edge detection
for file_name in uploaded.keys():
    # Load image in color (BGR format by default in OpenCV)
    img_bgr = cv2.imread(file_name)

    if img_bgr is None:
        print("Failed to load image.")
    else:
        # Convert to grayscale
        gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)

        # Apply Gaussian blur to reduce noise (recommended before Canny)
        blurred = cv2.GaussianBlur(gray, (5, 5), 1.4)

        # Apply Canny edge detector
        edges = cv2.Canny(blurred, threshold1=100, threshold2=200)

        # Display original and edge-detected image
        plt.figure(figsize=(10, 4))

        plt.subplot(1, 2, 1)
        plt.title("Original Image")
        plt.imshow(cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB))
        plt.axis('off')

        plt.subplot(1, 2, 2)
        plt.title("Canny Edge Detection")
```

```

plt.imshow(edges, cmap='gray')
plt.axis('off')

plt.tight_layout()
plt.show()

# Save and download the result
cv2.imwrite("canny_edge_output.png", edges)
files.download("canny_edge_output.png")

```

7)Write a code for obtaining histogram of the image Input: 24 bit RGB image Expected Output: Three separate histograms for individual RGB channels

```

# Import required libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Upload a 24-bit RGB image
uploaded = files.upload()

# Process the uploaded image
for file_name in uploaded.keys():
    # Read image in color (BGR format)
    img_bgr = cv2.imread(file_name)

    if img_bgr is None:
        print("Failed to load image.")
    else:
        # Convert to RGB for proper display and channel separation
        img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

        # Split into R, G, B channels
        r, g, b = cv2.split(img_rgb)

        # Calculate histograms
        hist_r = cv2.calcHist([r], [0], None, [256], [0, 256])

```

```

hist_g = cv2.calcHist([g], [0], None, [256], [0, 256])
hist_b = cv2.calcHist([b], [0], None, [256], [0, 256])

# Plot histograms
plt.figure(figsize=(10, 5))
plt.title("RGB Channel Histograms")
plt.plot(hist_r, color='red', label='Red Channel')
plt.plot(hist_g, color='green', label='Green Channel')
plt.plot(hist_b, color='blue', label='Blue Channel')
plt.xlabel('Pixel Value')
plt.ylabel('Frequency')
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()

```

8) Write a code for demonstrating histogram equalization in the image Input: 24 bit RGB image
 Expected Output: i) 24 bit RGB Equalized image ii) Original image histogram and equalized histogram

```

# Import required libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Upload an image
uploaded = files.upload()

# Process the uploaded image
for file_name in uploaded.keys():
    # Load image in BGR format
    img_bgr = cv2.imread(file_name)

    if img_bgr is None:
        print("Failed to load image.")
    else:
        # Convert BGR to RGB for display

```

```

img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

# Split RGB channels
r, g, b = cv2.split(img_rgb)

# Equalize each channel separately
r_eq = cv2.equalizeHist(r)
g_eq = cv2.equalizeHist(g)
b_eq = cv2.equalizeHist(b)

# Merge equalized channels back to RGB
img_eq_rgb = cv2.merge([r_eq, g_eq, b_eq])

# Function to plot histogram
def plot_histogram(channel, color, label):
    hist = cv2.calcHist([channel], [0], None, [256], [0, 256])
    plt.plot(hist, color=color, label=label)

# Display original and equalized images
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.title("Original Image")
plt.imshow(img_rgb)
plt.axis('off')

plt.subplot(1, 2, 2)
plt.title("Histogram Equalized Image")
plt.imshow(img_eq_rgb)
plt.axis('off')
plt.tight_layout()
plt.show()

# Plot original histograms
plt.figure(figsize=(10, 4))
plt.title("Original RGB Histograms")
plot_histogram(r, 'red', 'Red Channel')
plot_histogram(g, 'green', 'Green Channel')
plot_histogram(b, 'blue', 'Blue Channel')
plt.xlabel("Pixel Value")
plt.ylabel("Frequency")

```

```

plt.legend()
plt.grid()
plt.show()

# Plot equalized histograms
plt.figure(figsize=(10, 4))
plt.title("Equalized RGB Histograms")
plot_histogram(r_eq, 'red', 'Red Channel (Equalized)')
plot_histogram(g_eq, 'green', 'Green Channel (Equalized)')
plot_histogram(b_eq, 'blue', 'Blue Channel (Equalized)')
plt.xlabel("Pixel Value")
plt.ylabel("Frequency")
plt.legend()
plt.grid()
plt.show()

# Save and download the equalized image
img_eq_bgr = cv2.cvtColor(img_eq_rgb, cv2.COLOR_RGB2BGR)
cv2.imwrite("equalized_image.png", img_eq_bgr)
files.download("equalized_image.png")

```

9) Write a code for demonstrating Image transformation: Scaling Input: 24 bit RGB image
 Expected Output: i) 24 bit RGB image scaled by factor 2 ii) 24 bit RGB image scaled by factor 0.5

```

# Import required libraries
import cv2
import matplotlib.pyplot as plt
from google.colab import files

# Upload a 24-bit RGB image
uploaded = files.upload()

# Process uploaded image
for file_name in uploaded.keys():
    # Load image (in BGR format)
    img_bgr = cv2.imread(file_name)

```

```
if img_bgr is None:
    print("Failed to load image.")
else:
    # Convert BGR to RGB for display
    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

    # Get original dimensions
    height, width = img_rgb.shape[:2]

    # Scale up by factor of 2
    img_up = cv2.resize(img_rgb, (2 * width, 2 * height),
interpolation=cv2.INTER_LINEAR)

    # Scale down by factor of 0.5
    img_down = cv2.resize(img_rgb, (width // 2, height // 2),
interpolation=cv2.INTER_AREA)

    # Display all images
    plt.figure(figsize=(15, 5))

    plt.subplot(1, 3, 1)
    plt.title("Original Image")
    plt.imshow(img_rgb)
    plt.axis('off')

    plt.subplot(1, 3, 2)
    plt.title("Scaled Up (×2)")
    plt.imshow(img_up)
    plt.axis('off')

    plt.subplot(1, 3, 3)
    plt.title("Scaled Down (×0.5)")
    plt.imshow(img_down)
    plt.axis('off')

    plt.tight_layout()
    plt.show()

    # Save and download results
```

```

        cv2.imwrite("scaled_up.png", cv2.cvtColor(img_up,
cv2.COLOR_RGB2BGR))
        cv2.imwrite("scaled_down.png", cv2.cvtColor(img_down,
cv2.COLOR_RGB2BGR))
        files.download("scaled_up.png")
        files.download("scaled_down.png")

```

10)Write a code for demonstrating Image transformation: Translation Input: 24 bit RGB image
Expected Output: i) 24 bit RGB image shifted to right by 20 units ii) 24 bit RGB image shifted to downwards by 10 units

```

# Import required libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Upload a 24-bit RGB image
uploaded = files.upload()

# Process uploaded image
for file_name in uploaded.keys():
    # Load image in BGR format
    img_bgr = cv2.imread(file_name)

    if img_bgr is None:
        print("Failed to load image.")
    else:
        # Convert to RGB for display
        img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

        # Get image dimensions
        height, width = img_rgb.shape[:2]

        # Define translation matrices
        M_right = np.float32([[1, 0, 20], [0, 1, 0]]) # Shift right by
20
        M_down = np.float32([[1, 0, 0], [0, 1, 10]]) # Shift down by 10

```

```

# Apply affine transformation (translation)
img_shift_right = cv2.warpAffine(img_rgb, M_right, (width,
height))
img_shift_down = cv2.warpAffine(img_rgb, M_down, (width, height))

# Display all images
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.title("Original Image")
plt.imshow(img_rgb)
plt.axis('off')

plt.subplot(1, 3, 2)
plt.title("Shifted Right by 20px")
plt.imshow(img_shift_right)
plt.axis('off')

plt.subplot(1, 3, 3)
plt.title("Shifted Down by 10px")
plt.imshow(img_shift_down)
plt.axis('off')

plt.tight_layout()
plt.show()

# Save and download results
cv2.imwrite("translated_right.png", cv2.cvtColor(img_shift_right,
cv2.COLOR_RGB2BGR))
cv2.imwrite("translated_down.png", cv2.cvtColor(img_shift_down,
cv2.COLOR_RGB2BGR))
files.download("translated_right.png")
files.download("translated_down.png")

```

11) Write a code for demonstrating Image transformation: Rotation
Input: 24 bit RGB image
Expected Output: i) 24 bit RGB image rotated clockwise by 90 degree
ii) 24 bit RGB image rotated anti-clockwise by 90 degree

```

# Import required libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Upload a 24-bit RGB image
uploaded = files.upload()

# Process uploaded image
for file_name in uploaded.keys():
    # Load image in BGR format
    img_bgr = cv2.imread(file_name)

    if img_bgr is None:
        print("Failed to load image.")
    else:
        # Convert to RGB for display
        img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

        # Get image dimensions
        height, width = img_rgb.shape[:2]

        # Rotation matrix for clockwise 90 degrees
        M_clockwise = cv2.getRotationMatrix2D((width / 2, height / 2),
-90, 1)

        # Rotation matrix for anti-clockwise 90 degrees
        M_anticlockwise = cv2.getRotationMatrix2D((width / 2, height / 2),
90, 1)

        # Perform rotation
        img_rot_clockwise = cv2.warpAffine(img_rgb, M_clockwise, (width,
height))
        img_rot_anticlockwise = cv2.warpAffine(img_rgb, M_anticlockwise,
(width, height))

        # Display all images
        plt.figure(figsize=(15, 5))

```

```

plt.subplot(1, 3, 1)
plt.title("Original Image")
plt.imshow(img_rgb)
plt.axis('off')

plt.subplot(1, 3, 2)
plt.title("Rotated Clockwise (90°)")
plt.imshow(img_rot_clockwise)
plt.axis('off')

plt.subplot(1, 3, 3)
plt.title("Rotated Anti-clockwise (90°)")
plt.imshow(img_rot_anticlockwise)
plt.axis('off')

plt.tight_layout()
plt.show()

# Save and download results
cv2.imwrite("rotated_clockwise.png",
cv2.cvtColor(img_rot_clockwise, cv2.COLOR_RGB2BGR))
cv2.imwrite("rotated_anticlockwise.png",
cv2.cvtColor(img_rot_anticlockwise, cv2.COLOR_RGB2BGR))
files.download("rotated_clockwise.png")
files.download("rotated_anticlockwise.png")

```

12) Write a code for demonstrating Image Compression. Apply Jpeg compression with suitable factor. Display Original Image and Decompressed Image Input: 24 bit RGB image Expected Output: PSNR value comparing Original Image and Decompressed Image

```

# Import required libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
import math

```

```

# Function to calculate PSNR
def calculate_psnr(original, compressed):
    mse = np.mean((original - compressed) ** 2)
    if mse == 0:
        return 100 # No noise, perfect match
    max_pixel = 255.0
    psnr = 20 * math.log10(max_pixel / math.sqrt(mse))
    return psnr

# Upload a 24-bit RGB image
uploaded = files.upload()

# Process uploaded image
for file_name in uploaded.keys():
    # Load image in BGR format
    img_bgr = cv2.imread(file_name)

    if img_bgr is None:
        print("Failed to load image.")
    else:
        # Convert to RGB for display
        img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

        # Set JPEG compression quality (lower is more compressed)
        quality_factor = 30 # You can adjust this value from 0 to 100
        (100 is best quality)

        # Compress the image (JPEG compression)
        encode_param = [int(cv2.IMWRITE_JPEG_QUALITY), quality_factor]
        _, img_compressed = cv2.imencode('.jpg', img_rgb, encode_param)

        # Decompress the image back to RGB format
        img_decompressed = cv2.imdecode(img_compressed, cv2.IMREAD_COLOR)
        img_decompressed_rgb = cv2.cvtColor(img_decompressed,
cv2.COLOR_BGR2RGB)

        # Calculate PSNR
        psnr_value = calculate_psnr(img_rgb, img_decompressed_rgb)

        # Display Original and Decompressed Images

```

```

plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.title("Original Image")
plt.imshow(img_rgb)
plt.axis('off')

plt.subplot(1, 3, 2)
plt.title(f"Compressed and Decompressed Image (JPEG)\nPSNR:
{psnr_value:.2f} dB")
plt.imshow(img_decompressed_rgb)
plt.axis('off')

plt.tight_layout()
plt.show()

# Print PSNR value
print(f"PSNR value between original and decompressed image:
{psnr_value:.2f} dB")

```

13) Write a code for demonstrating Image Transformation from spatial domain into frequency domain. Apply DCT transformation. Display Original Image and Image retrieved after inverse (IDCT) transform Input: 24 bit RGB image Expected Output: PSNR value comparing Original Image and IDCT Image

```

# Import required libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
import math

# Function to calculate PSNR
def calculate_psnr(original, decompressed):
    mse = np.mean((original - decompressed) ** 2)
    if mse == 0:
        return 100 # No noise, perfect match
    max_pixel = 255.0
    psnr = 20 * math.log10(max_pixel / math.sqrt(mse))

```

```

    return psnr

# Upload a 24-bit RGB image
uploaded = files.upload()

# Process uploaded image
for file_name in uploaded.keys():
    # Load image in BGR format
    img_bgr = cv2.imread(file_name)

    if img_bgr is None:
        print("Failed to load image.")
    else:
        # Convert to RGB for display
        img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

        # Apply DCT to each channel (RGB channels separately)
        img_dct = np.zeros_like(img_rgb, dtype=np.float32)

        for i in range(3): # Apply DCT for each channel (R, G, B)
            img_dct[:, :, i] = cv2.dct(np.float32(img_rgb[:, :, i]))

        # Apply Inverse DCT (IDCT)
        img_idct = np.zeros_like(img_rgb, dtype=np.float32)
        for i in range(3): # Apply IDCT for each channel (R, G, B)
            img_idct[:, :, i] = cv2.idct(img_dct[:, :, i])

        # Clip values to ensure they are in valid range [0, 255]
        img_idct = np.clip(img_idct, 0, 255).astype(np.uint8)

        # Calculate PSNR between original and IDCT image
        psnr_value = calculate_psnr(img_rgb, img_idct)

        # Display Original and IDCT Images
        plt.figure(figsize=(15, 5))

        plt.subplot(1, 3, 1)
        plt.title("Original Image")
        plt.imshow(img_rgb)
        plt.axis('off')

```

```

plt.subplot(1, 3, 2)
plt.title(f"IDCT Image\nPSNR: {psnr_value:.2f} dB")
plt.imshow(img_idct)
plt.axis('off')

plt.tight_layout()
plt.show()

# Print PSNR value
print(f"PSNR value between original and IDCT image:
{psnr_value:.2f} dB")

```

14) Write a code for demonstrating Image Transformation from spatial domain into frequency domain. Apply DWT transformation. Display Original Image and Image retrieved after inverse (IDWT) transform Input: 24 bit RGB image Expected Output: PSNR value comparing Original Image and IDWT Image

```

# Step 1: Install required libraries
!pip install opencv-python-headless matplotlib pywavelets

# Step 2: Import libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt
import pywt
from google.colab import files

# Step 3: Upload image
uploaded = files.upload()

# Step 4: Define PSNR function
def compute_psnr(original, reconstructed):
    mse = np.mean((original - reconstructed) ** 2)
    if mse == 0:
        return float('inf')
    PIXEL_MAX = 255.0
    return 20 * np.log10(PIXEL_MAX / np.sqrt(mse))

```

```

# Step 5: Process the image
for filename in uploaded.keys():
    # Load the image in BGR format
    img_bgr = cv2.imread(filename)

    if img_bgr is None:
        print("Failed to load image.")
    else:
        # Convert to RGB
        img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

        # Initialize list to store reconstructed channels
        reconstructed_channels = []

        # Apply DWT and IDWT to each channel
        for i in range(3): # For R, G, B channels
            channel = img_rgb[:, :, i]

            # Apply 2D DWT (using Haar wavelet)
            coeffs2 = pywt.dwt2(channel, 'haar')
            cA, (cH, cV, cD) = coeffs2

            # Apply inverse DWT
            reconstructed = pywt.idwt2((cA, (cH, cV, cD)), 'haar')
            reconstructed = np.clip(reconstructed, 0, 255)
            reconstructed = reconstructed.astype(np.uint8)

            # Resize to match original size if needed (rounding in DWT may
            # reduce size)
            reconstructed = cv2.resize(reconstructed, (channel.shape[1],
            channel.shape[0]))

            reconstructed_channels.append(reconstructed)

        # Merge channels back
        reconstructed_rgb = cv2.merge(reconstructed_channels)

        # Calculate PSNR
        psnr = compute_psnr(img_rgb.astype(np.float32),
        reconstructed_rgb.astype(np.float32))

```

```

# Display Original and Reconstructed images
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.title("Original Image")
plt.imshow(img_rgb)
plt.axis('off')

plt.subplot(1, 2, 2)
plt.title(f"Reconstructed Image (IDWT)\nPSNR: {psnr:.2f} dB")
plt.imshow(reconstructed_rgb)
plt.axis('off')

plt.tight_layout()
plt.show()

```

15) Write a code for applying threshold in the image. Input: Gray scale 8 bit image and Threshold Value Expected output: Binary Image after applying threshold

```

# Import required libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Function to apply threshold
def apply_threshold(image, threshold_value):
    # Apply binary thresholding
    _, binary_image = cv2.threshold(image, threshold_value, 255,
cv2.THRESH_BINARY)
    return binary_image

# Upload the grayscale 8-bit image
uploaded = files.upload()

# Process uploaded image
for file_name in uploaded.keys():
    # Load the image in grayscale (8-bit)

```

```

img_gray = cv2.imread(file_name, cv2.IMREAD_GRAYSCALE)

if img_gray is None:
    print("Failed to load image.")
else:
    # Specify the threshold value (you can change this value)
    threshold_value = 127 # Example threshold value

    # Apply the threshold
    binary_image = apply_threshold(img_gray, threshold_value)

    # Display the original and binary image
    plt.figure(figsize=(10, 5))

    # Display original grayscale image
    plt.subplot(1, 2, 1)
    plt.title("Original Grayscale Image")
    plt.imshow(img_gray, cmap='gray')
    plt.axis('off')

    # Display binary image after thresholding
    plt.subplot(1, 2, 2)
    plt.title("Binary Image after Thresholding")
    plt.imshow(binary_image, cmap='gray')
    plt.axis('off')

    plt.tight_layout()
    plt.show()

```

16) Write a code for demonstrating Image Normalization. Input: 24 bit RGB image Expected output: Normalized 24 bit RGB image

```

# Import required libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Function to normalize image
def normalize_image(image):

```

```
# Normalize the image to range [0, 1] by dividing by 255
normalized_image = image / 255.0
return normalized_image

# Upload the 24-bit RGB image
uploaded = files.upload()

# Process uploaded image
for file_name in uploaded.keys():
    # Load image in BGR format (OpenCV uses BGR by default)
    img_bgr = cv2.imread(file_name)

    if img_bgr is None:
        print("Failed to load image.")
    else:
        # Convert the image to RGB (matplotlib uses RGB by default)
        img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

        # Normalize the image to range [0, 1]
        img_normalized = normalize_image(img_rgb)

        # Convert back to [0, 255] range for visualization and display
        img_normalized_display = (img_normalized * 255).astype(np.uint8)

        # Display the original and normalized image
        plt.figure(figsize=(15, 5))

        plt.subplot(1, 2, 1)
        plt.title("Original RGB Image")
        plt.imshow(img_rgb)
        plt.axis('off')

        plt.subplot(1, 2, 2)
        plt.title("Normalized RGB Image")
        plt.imshow(img_normalized_display)
        plt.axis('off')

        plt.tight_layout()
        plt.show()
```

17) Write a code for demonstrating Intensity slicing in the Image. Apply Gray-level slicing:- Highlighting specific range of intensity values. Input: Gray scale 8 bit image Expected output: Gray scale 8 bit image i) With preserving the background ii) With Non-preserving the background

```
# Import required libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Function to apply gray-level slicing (highlighting specific intensity range)
def intensity_slicing_preserve_background(image, lower_limit, upper_limit):
    # Create a mask where the pixel intensity is within the given range
    mask = (image >= lower_limit) & (image <= upper_limit)

    # Create the sliced image, where only pixels within the range are preserved
    sliced_image = image.copy()
    sliced_image[~mask] = 0 # Set pixels outside the range to 0
    (background remains intact)

    return sliced_image

def intensity_slicing_no_preserve_background(image, lower_limit, upper_limit):
    # Create a mask where the pixel intensity is within the given range
    mask = (image >= lower_limit) & (image <= upper_limit)

    # Create the sliced image, where only pixels within the range are kept as white
    sliced_image = np.zeros_like(image) # Initialize as black
    sliced_image[mask] = 255 # Set the pixels within the range to white

    return sliced_image

# Upload the grayscale 8-bit image
uploaded = files.upload()
```

```

# Process uploaded image
for file_name in uploaded.keys():
    # Load the image in grayscale (8-bit)
    img_gray = cv2.imread(file_name, cv2.IMREAD_GRAYSCALE)

    if img_gray is None:
        print("Failed to load image.")
    else:
        # Define the intensity range (you can change these values)
        lower_limit = 100 # Lower intensity range value
        upper_limit = 150 # Upper intensity range value

        # Apply intensity slicing with preserving the background
        sliced_image_preserve =
intensity_slicing_preserve_background(img_gray, lower_limit, upper_limit)

        # Apply intensity slicing without preserving the background
        sliced_image_no_preserve =
intensity_slicing_no_preserve_background(img_gray, lower_limit,
upper_limit)

        # Display the original and sliced images
        plt.figure(figsize=(15, 5))

        # Display original grayscale image
        plt.subplot(1, 3, 1)
        plt.title("Original Grayscale Image")
        plt.imshow(img_gray, cmap='gray')
        plt.axis('off')

        # Display intensity sliced image (preserving background)
        plt.subplot(1, 3, 2)
        plt.title("Sliced Image (Preserve Background)")
        plt.imshow(sliced_image_preserve, cmap='gray')
        plt.axis('off')

        # Display intensity sliced image (non-preserving background)
        plt.subplot(1, 3, 3)
        plt.title("Sliced Image (No Background)")

```

```
plt.imshow(sliced_image_no_preserve, cmap='gray')
plt.axis('off')

plt.tight_layout()
plt.show()
```

18)Write a code for demonstrating Image arithmetic Input: Two Gray scale 8 bit images
Expected output: Gray scale 8 bit image by applying i) Addition of two Images ii) Subtraction of two Images

```
# Import required libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Upload first image
print("Upload the FIRST grayscale image:")
uploaded1 = files.upload()
img1_path = list(uploaded1.keys())[0]
img1 = cv2.imread(img1_path, cv2.IMREAD_GRAYSCALE)

# Upload second image
print("Upload the SECOND grayscale image:")
uploaded2 = files.upload()
img2_path = list(uploaded2.keys())[0]
img2 = cv2.imread(img2_path, cv2.IMREAD_GRAYSCALE)

# Resize second image to match the first if sizes are different
if img1.shape != img2.shape:
    img2 = cv2.resize(img2, (img1.shape[1], img1.shape[0]))

# Perform image addition and subtraction using OpenCV
added_image = cv2.add(img1, img2)
subtracted_image = cv2.subtract(img1, img2)

# Display all images
plt.figure(figsize=(15, 6))
```

```

plt.subplot(2, 2, 1)
plt.title("First Grayscale Image")
plt.imshow(img1, cmap='gray')
plt.axis('off')

plt.subplot(2, 2, 2)
plt.title("Second Grayscale Image")
plt.imshow(img2, cmap='gray')
plt.axis('off')

plt.subplot(2, 2, 3)
plt.title("Added Image")
plt.imshow(added_image, cmap='gray')
plt.axis('off')

plt.subplot(2, 2, 4)
plt.title("Subtracted Image")
plt.imshow(subtracted_image, cmap='gray')
plt.axis('off')

plt.tight_layout()
plt.show()

```

19) Write a code for demonstrating Image arithmetic. Enhance any one color channel by multiplying enhancement factor. Input: 24 bit RGB image Expected output: 24 bit RGB Enhanced image

```

# Import required libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Upload a 24-bit RGB image
print("Upload a 24-bit RGB image:")
uploaded = files.upload()

# Load the image
for file_name in uploaded.keys():

```

```

# Read the image in BGR format
img_bgr = cv2.imread(file_name)

if img_bgr is None:
    print("Failed to load image.")
else:
    # Convert BGR to RGB
    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

    # Enhancement parameters
    enhancement_channel = 'R' # Choose 'R', 'G', or 'B'
    enhancement_factor = 1.5 # Multiply channel by this factor

    # Split channels
    R, G, B = cv2.split(img_rgb)

    # Enhance the selected channel
    if enhancement_channel == 'R':
        R = np.clip(R * enhancement_factor, 0, 255).astype(np.uint8)
    elif enhancement_channel == 'G':
        G = np.clip(G * enhancement_factor, 0, 255).astype(np.uint8)
    elif enhancement_channel == 'B':
        B = np.clip(B * enhancement_factor, 0, 255).astype(np.uint8)

    # Merge enhanced channels
    enhanced_img = cv2.merge([R, G, B])

    # Display original and enhanced images
    plt.figure(figsize=(12, 5))

    plt.subplot(1, 2, 1)
    plt.title("Original Image")
    plt.imshow(img_rgb)
    plt.axis('off')

    plt.subplot(1, 2, 2)
    plt.title(f"Enhanced Image ({enhancement_channel} × {enhancement_factor})")
    plt.imshow(enhanced_img)
    plt.axis('off')

```

```
plt.tight_layout()
plt.show()
```

20)Write a code for demonstrating Image smoothening/Low pass filtering. Input: 24 bit RGB Noisy image Expected output: 24 bit RGB smooth image

```
# Import required libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Upload a noisy 24-bit RGB image
print("Upload a noisy RGB image:")
uploaded = files.upload()

# Load and process the uploaded image
for file_name in uploaded.keys():
    # Read the image in BGR format
    img_bgr = cv2.imread(file_name)

    if img_bgr is None:
        print("Failed to load image.")
    else:
        # Convert BGR to RGB for display
        img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

        # Apply different smoothing techniques
        avg_blur = cv2.blur(img_rgb, (5, 5)) # Average filter
        gaussian_blur = cv2.GaussianBlur(img_rgb, (5, 5), 0) # Gaussian
filter
        median_blur = cv2.medianBlur(img_rgb, 5) # Median filter

        # Display the original and smoothed images
        plt.figure(figsize=(12, 8))

        plt.subplot(2, 2, 1)
```

```

plt.title("Original Noisy Image")
plt.imshow(img_rgb)
plt.axis('off')

plt.subplot(2, 2, 2)
plt.title("Average Blur")
plt.imshow(avg_blur)
plt.axis('off')

plt.subplot(2, 2, 3)
plt.title("Gaussian Blur")
plt.imshow(gaussian_blur)
plt.axis('off')

plt.subplot(2, 2, 4)
plt.title("Median Blur")
plt.imshow(median_blur)
plt.axis('off')

plt.tight_layout()
plt.show()

```

Convert image into transform domain

Objective: Learn DFT representations and their support for compression

```

import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Step 3: Upload image
uploaded = files.upload()

# Step 4: Apply DFT
for filename in uploaded.keys():
    # Load image in grayscale
    img = cv2.imread(filename, cv2.IMREAD_GRAYSCALE)

    if img is None:

```

```

    print("Error loading image.")
else:
    # Get DFT and shift zero freq to center
    dft = cv2.dft(np.float32(img), flags=cv2.DFT_COMPLEX_OUTPUT)
    dft_shift = np.fft.fftshift(dft)

    # Compute magnitude spectrum
    magnitude = cv2.magnitude(dft_shift[:, :, 0], dft_shift[:, :, 1])
    magnitude_log = 20 * np.log(magnitude + 1) # log for visualization

    # Reconstruct image using inverse DFT
    dft_ishift = np.fft.ifftshift(dft_shift)
    img_back = cv2.idft(dft_ishift)
    img_back = cv2.magnitude(img_back[:, :, 0], img_back[:, :, 1])

    # Step 5: Show results
    plt.figure(figsize=(15, 5))

    plt.subplot(1, 3, 1)
    plt.title("Original Image")
    plt.imshow(img, cmap='gray')
    plt.axis('off')

    plt.subplot(1, 3, 2)
    plt.title("DFT Magnitude Spectrum")
    plt.imshow(magnitude_log, cmap='gray')
    plt.axis('off')

    plt.subplot(1, 3, 3)
    plt.title("Reconstructed Image (IDFT)")
    plt.imshow(img_back, cmap='gray')
    plt.axis('off')

    plt.tight_layout()
    plt.show()

```

Apply spatial and frequency domain filters

Objective: Apply image de-noising, smoothing, sharpening etc.

Step 1: Install required libraries

```
!pip install opencv-python-headless matplotlib numpy
```

```
# Step 2: Import libraries
```

```
import cv2
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from google.colab import files
```

```
# Step 3: Upload an image
```

```
uploaded = files.upload()
```

```
# Step 4: Read and process the image
```

```
for filename in uploaded.keys():
```

```
    # Read image (BGR) and convert to RGB
```

```
    img_bgr = cv2.imread(filename)
```

```
    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
```

```
    gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
```

```
# ----- SPATIAL DOMAIN FILTERING -----
```

```
# 1. Smoothing (Average filter)
```

```
smooth = cv2.blur(img_rgb, (5, 5))
```

```
# 2. Sharpening using kernel
```

```
sharpening_kernel = np.array([[0, -1, 0],
```

```
                             [-1, 5, -1],
```

```
                             [0, -1, 0]])
```

```
sharpened = cv2.filter2D(img_rgb, -1, sharpening_kernel)
```

```
# 3. Gaussian Denoising
```

```
gaussian = cv2.GaussianBlur(img_rgb, (5, 5), 1)
```

```
# ----- FREQUENCY DOMAIN FILTERING -----
```

```
# Convert grayscale to float32
```

```
dft = cv2.dft(np.float32(gray), flags=cv2.DFT_COMPLEX_OUTPUT)
```

```
dft_shift = np.fft.fftshift(dft)
```

```

# Create masks for low-pass and high-pass filters
rows, cols = gray.shape
crow, ccol = rows // 2 , cols // 2

mask_low = np.zeros((rows, cols, 2), np.uint8)
mask_low[crow-30:crow+30, ccol-30:ccol+30] = 1

mask_high = 1 - mask_low

# Apply low-pass filter
fshift_low = dft_shift * mask_low
f_ishift_low = np.fft.ifftshift(fshift_low)
img_back_low = cv2.idft(f_ishift_low)
img_back_low = cv2.magnitude(img_back_low[:, :, 0], img_back_low[:, :, 1])

# Apply high-pass filter
fshift_high = dft_shift * mask_high
f_ishift_high = np.fft.ifftshift(fshift_high)
img_back_high = cv2.idft(f_ishift_high)
img_back_high = cv2.magnitude(img_back_high[:, :, 0], img_back_high[:, :, 1])

# ----- DISPLAY RESULTS -----
plt.figure(figsize=(16, 10))

plt.subplot(3, 3, 1)
plt.title("Original RGB")
plt.imshow(img_rgb)
plt.axis('off')

plt.subplot(3, 3, 2)
plt.title("Smoothed (Blur)")
plt.imshow(smooth)
plt.axis('off')

plt.subplot(3, 3, 3)
plt.title("Sharpened")

```

```
plt.imshow(sharpened)
plt.axis('off')
```

```
plt.subplot(3, 3, 4)
plt.title("Gaussian Denoising")
plt.imshow(gaussian)
plt.axis('off')
```

```
plt.subplot(3, 3, 5)
plt.title("Grayscale")
plt.imshow(gray, cmap='gray')
plt.axis('off')
```

```
plt.subplot(3, 3, 6)
plt.title("Low-Pass (Freq Domain)")
plt.imshow(np.uint8(img_back_low), cmap='gray')
plt.axis('off')
```

```
plt.subplot(3, 3, 7)
plt.title("High-Pass (Freq Domain)")
plt.imshow(np.uint8(img_back_high), cmap='gray')
plt.axis('off')
```

```
plt.tight_layout()
plt.show()
```

Apply Morphological Operations

Objective: Calculate image matrices with mathematical and logical operations

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
```

Step 3: Upload a binary or grayscale image

```

uploaded = files.upload()

# Step 4: Read and binarize image
for filename in uploaded.keys():
    img = cv2.imread(filename, cv2.IMREAD_GRAYSCALE)

    if img is None:
        print("Image loading failed.")
    else:
        # Convert to binary (thresholding)
        _, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)

        # Define a structuring element (kernel)
        kernel = np.ones((5, 5), np.uint8)

        # Apply Morphological Operations
        erosion = cv2.erode(binary, kernel, iterations=1)
        dilation = cv2.dilate(binary, kernel, iterations=1)
        opening = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)
        closing = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel)

        # Display results
        plt.figure(figsize=(15, 8))

        plt.subplot(2, 3, 1)
        plt.title("Original Binary Image")
        plt.imshow(binary, cmap='gray')
        plt.axis('off')

        plt.subplot(2, 3, 2)
        plt.title("Erosion")
        plt.imshow(erosion, cmap='gray')
        plt.axis('off')

        plt.subplot(2, 3, 3)
        plt.title("Dilation")
        plt.imshow(dilation, cmap='gray')
        plt.axis('off')

        plt.subplot(2, 3, 4)
        plt.title("Opening (Remove Noise)")
        plt.imshow(opening, cmap='gray')
        plt.axis('off')

```

```
plt.subplot(2, 3, 5)  
plt.title("Closing (Close Holes)")  
plt.imshow(closing, cmap='gray')  
plt.axis('off')
```

```
plt.tight_layout()  
plt.show()
```